Protecting the ALU Logo on the Ethereum Blockchain

Assignment 1 – Cryptocurrency and Digital Assets · Student: [YOUR FULL NAME]

Abstract

This project registers the official ALU logo on an Ethereum blockchain as an ERC-721 NFT that stores the file's SHA-256 fingerprint, its registrant and a timestamp, so anyone can check if a logo file is genuine. A second ERC-20 contract splits ownership of the logo into 1,000,000 ALUT share tokens. It solves the problem of proving authenticity and ownership of a digital asset without trusting a single database holder.

Introduction

ALU's stated mission is to develop the next generation of ethical, entrepreneurial leaders who will shape Africa and the world. That mission depends on trust: students, employers and partners must be able to rely on anything carrying ALU's name, and a forged logo on a fake certificate or programme undermines it. The project also applies the entrepreneurial, problem-solving approach ALU teaches to a real problem facing the university.

A logo is easy to copy, edit and falsely claim. A normal database is controlled by one administrator who can edit or delete records silently, and a copyright registration proves a claim to a legal office but cannot be checked instantly by a stranger, says nothing about whether a given file was altered, and is slow and jurisdiction-specific. A blockchain record is public, timestamped, tamper-evident and verifiable by anyone in seconds with no permission. Registering the file's hash proves *this exact file was claimed by this wallet at this time*, and any modified fake yields a different hash.

Blockchain Setup

I installed Node.js, then created an npm project and installed Hardhat 2.29.1, the Hardhat Toolbox (ethers, chai matchers) and OpenZeppelin Contracts 5.1.0. `hardhat.config.js` selects Solidity 0.8.28. Hardhat compiles the contracts, runs the tests on an in-memory chain, and `npx hardhat node` starts a local Ethereum node at `http://127.0.0.1:8545` with 20 funded test accounts of 10,000 fake ETH each.

Local blockchain: a private Ethereum network on your own computer. Developers use it because it is free (no real ETH), instant (blocks mined on demand), resettable, and mistakes cannot lose real money or become permanent on a public chain.

Wallet address: a 42-character hex identifier (e.g. `0xf39F…2266`) derived from a public key. It represents an account that can hold ETH, NFTs and tokens and sign transactions with its private key.

```
% npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/
Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Account #6: 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (10000 ETH)
Account #7: 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (10000 ETH)
Account #8: 0x23618e81E3f5cdF7f54C3d65f7FBc0aBf5B21E8f (10000 ETH)
Account #9: 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000 ETH)
Account #10: 0xBcd4042DE499D14e55001CcbB24a551F3b954096 (10000 ETH)
Account #11: 0x71bE63f3384f5fb98995898A86B02Fb2426c5788 (10000 ETH)
Account #12: 0xFABB0ac9d68B0B445fB7357272Ff202C5651694a (10000 ETH)
Account #13: 0x1CBd3b2770909D4e10f157cABC84C7264073C9Ec (10000 ETH)
Account #14: 0xdF3e18d64BC6A983f673Ab319CCaE4f1a57C7097 (10000 ETH)
Account #15: 0xcd3B766CCDd6AE721141F452C550Ca635964ce71 (10000 ETH)
Account #16: 0x2546BcD3c84621e976D8185a91A922aE77ECEc30 (10000 ETH)
Account #17: 0xbDA5747bFD65F08deb54cb465eB87D40e51B197E (10000 ETH)
Account #18: 0xdD2FD4581271e230360230F9337D5c0430Bf44C0 (10000 ETH)
Account #19: 0x8626f6940E2eb28930eFb4CeF49B2d1F2C9C1199 (10000 ETH)
```

Figure 1: local node and its test wallet addresses (private-key lines omitted).

Smart Contract Explanation

Generating the hash. I downloaded the logo from alueducation.com (`alu-logo.png` , 500×188) and ran `shasum -a 256 alu-logo.png` . The deploy script and tests recompute the same value with Node's `crypto` module.

```
% shasum -a 256 alu-logo.png
727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560  alu-logo.png
```

Figure 2: SHA-256 hash of the ALU logo file.

ALUAssetRegistry (ERC-721). It defines `AssetMetadata` (name, file type, content hash as bytes32, registrant, timestamp), a mapping from token ID to metadata and a mapping from hash to bool.

- `registerAsset(name, fileType, contentHash)` – reverts with "Asset already registered: duplicate content hash" if the hash is known; otherwise increments the counter, stores the metadata, marks the hash used, mints the NFT to the caller with `_safeMint` , emits `AssetRegistered` and returns the token ID. State is written before minting to avoid re-entrancy problems.
- `verifyLogoIntegrity(tokenId, contentHash)` – a view function comparing the supplied hash to the stored one; returns `(true, "Logo is authentic.")` or `(false, "Warning: logo does not match.")` . It costs no gas when called off-chain.
- `getAsset(tokenId)` – a view function returning the whole struct.

What is a smart contract? Code stored and executed on the blockchain by every node, whose rules and state nobody can secretly change. A normal server program runs on a machine one company controls, which can alter

code or data at will; a smart contract's execution is public, deterministic and enforced by the network.

NFT and ERC-721. A non-fungible token is a unique on-chain token with its own ID. ERC-721 is right for a logo because the logo is one unique, indivisible asset: each token ID is distinct and has a single owner and attached metadata, unlike interchangeable ERC-20 coins.

SHA-256. A one-way function that turns any file into a fixed 256-bit fingerprint. Because of the avalanche effect, changing one pixel changes the input bytes, and the output changes completely and unpredictably (about half the bits flip), so edits cannot hide and the original cannot be reconstructed from the hash.

Immutability. Data is stored in blocks linked by hashes and replicated on thousands of nodes. Changing an old record would alter that block's hash and break every later block, and would need to be accepted by the majority of the network; the contract also has no function that edits or deletes a record.

ALULogoToken (ERC-20). Name "ALU Logo Token", symbol ALUT, fixed supply of 1,000,000 minted in the constructor to the owner address. `distributeShares(recipient, amount)` is `onlyOwner`, requires `amount > 0` and transfers tokens from the owner. `ownershipPercentage(account)` returns `balance × 100 / totalSupply` as a whole number (500,000 tokens → 50). `deploy.js` deploys the registry, registers the logo, then deploys the token to the deployer.

```
% npx hardhat run scripts/deploy.js --network localhost
Deploying with account: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
ALUAssetRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
ALU logo registered as token ID 1
SHA-256 content hash: 0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
ALULogoToken deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
Owner ALUT balance: 1000000
```

Figure 3: deployment to the local node.

Testnet deployment (Sepolia).

[AFTER DEPLOYING: paste the Etherscan links for ALUAssetRegistry and ALULogoToken on sepolia.etherscan.io]

Tokenisation

Tokenisation is representing ownership of an asset as digital tokens on a blockchain so it can be split, tracked and transferred. For the ALU logo, ALU as the original owner starts with all 1,000,000 ALUT and could give shares to faculty who help develop the brand, student representatives, the administration and partner institutions, rewarding contribution and giving stakeholders a formal stake. ERC-721 is for one unique item (the logo registration); ERC-20 is for many identical, divisible units (shares), which is why I used ERC-721 for registration and ERC-20 for ownership.

If a faculty member holds 100,000 of 1,000,000 ALUT, they own 10% of the logo's tokenised ownership. In practice that is a provable, transferable 10% stake, and could carry voting or revenue-share rights if ALU defined them (the token alone does not grant legal rights; those would need a legal framework).

Test Results

```
% npx hardhat test
  ALU Logo system
    ALUAssetRegistry (ERC-721)
      ✔ registers the ALU logo and returns a token ID
      ✔ rejects registering the same hash twice
      ✔ verifyLogoIntegrity returns true for the correct hash
      ✔ verifyLogoIntegrity returns false for an incorrect hash
      ✔ getAsset returns the correct name and file type
    ALULogoToken (ERC-20)
      ✔ mints the full 1,000,000 ALUT supply to the logo owner
      ✔ distributeShares transfers tokens to a recipient (owner only)
      ✔ ownershipPercentage returns the correct percentage


  8 passing (511ms)
```

Figure 4: all eight automated tests passing.

Conclusion

I learned how hashing gives a tamper-evident fingerprint, how ERC-721 and ERC-20 serve different purposes, and how OpenZeppelin and Hardhat fit together in testing. With more time I would store the logo on IPFS and add `tokenURI`, restrict registration to ALU-approved wallets, and build a front-end (the next assignment's dApp).

GitHub Link

https://github.com/goldensash1/Cryptocurrency-and-Digital-Assets

Video

[PASTE YOUR VIDEO LINK]

Question 1: Exploring a Cryptocurrency Exchange

DRAFT from public information, not from a real account. Sign up yourself, correct anything that differs, rewrite in your own words, add your screenshots, then delete this box.

Step 1: Creating an account. Coinbase asks for an email address, password and phone number, then your full name, date of birth and home address. To verify your identity you upload a photo of a government ID, and sometimes a selfie. Exchanges do this so criminals cannot easily use crypto to hide stolen money or fund terrorism. This is called Know Your Customer (KYC). In the United States it comes from the Bank Secrecy Act, enforced by FinCEN, and many countries follow similar anti-money-laundering rules based on Financial Action Task Force standards.

[SCREENSHOT: sign-up / verification screen, with personal details hidden]

Step 2: Exploring the platform. The dashboard shows your balance and market prices, the wallet section shows what you hold, and the trading area is where you buy and sell. Three listed coins:

Bitcoin (BTC) is the first cryptocurrency, used mainly to store value and send payments without a bank. What sets it apart is its fixed limit of 21 million coins.

Ethereum (ETH) is the coin of a blockchain that runs smart contracts, used to pay transaction fees and power applications such as NFTs. It is different because it is programmable, which is why this project is built on it.

USD Coin (USDC) is a stablecoin, used for moving dollars quickly and for parking value between trades. Unlike the other two, it aims to stay worth one US dollar, backed by reserves.

[SCREENSHOT: dashboard or asset list]

Step 3: Adding a payment method. In the payment settings you choose "add payment method". A debit card needs the card number, expiry date, security code and billing address. A bank account needs your account details, or a login through a linked banking service. A debit card is faster because the purchase happens almost immediately. A bank transfer is usually cheaper, but can take a few days to settle. So a card suits urgency, and a transfer suits saving on fees.

Step 4: Simulating a purchase. A market order buys immediately at the best available price, so the trade is certain but the exact price is not. A limit order only buys at a price you set or better, so the price is certain but it may never fill. I chose a market order because $100 is a small amount, and for a beginner speed and simplicity matter more than a tiny price difference. When I checked, Ethereum was about $2,261. Assuming total fees of about 1.5%, $1.50 goes to fees and $98.50 buys about 0.0436 ETH. [Replace with the exact amount and fee from your order preview.] I stopped at the preview, so no money was spent.

[SCREENSHOT: order preview showing the amount and fees, not confirmed]

Step 5: The transaction record. The history lists each transaction's type, asset, amount, value in dollars, fees, date and status. A transaction hash is a unique ID, itself a string of letters and numbers, given to a transaction on the blockchain. It matters because anyone can look it up on a block explorer to prove that the transfer happened, when it happened and whether it was confirmed. Trades inside an exchange may only appear in its own records, while withdrawals to an outside wallet get a hash.

Step 6: Security features. Two-factor authentication requires a second code from your phone or a security key when you log in. It matters because a stolen password alone is then not enough to enter the account. Withdrawal limits cap how much can leave an account in a period. They matter because crypto transfers usually cannot be reversed, so a limit reduces the damage if someone does break in.

[SCREENSHOT: security settings page]